NGINX One-Click Renewal Use Case - Keyfactor Command v25.5 Reference Implementation Guide

Audience

This guide is written for a junior engineer who has some Windows and Linux experience but is new to PKI, certificates, Keyfactor Command, Universal Orchestrator, and Linux certificate lifecycle automation.

The goal is to walk you through the implementation command by command, screen by screen, and decision by decision so you can build a repeatable one-click certificate renewal demo for nginx.

What You Will Build

You will build a working demo where Keyfactor Command renews an nginx TLS certificate, deploys the renewed certificate and private key to a Linux server, promotes the new certificate into nginx's active certificate paths, reloads nginx, and updates the visible web page so the audience can confirm the certificate changed.

The final demo flow looks like this:

Open the nginx Keyfactor demo page
Show the current certificate serial number and validity period
Run nginx-cert-status on the Linux server
Perform a one-click renewal from Keyfactor Command
Watch the RFPEM management job complete
Refresh the web page
Run nginx-cert-status again
Confirm the staged certificate and active nginx certificate both changed

Prerequisites Already Assumed Complete

This guide starts after the following are already installed and integrated:

- Keyfactor Command v25.5 is installed and operational.
- Keyfactor Universal Orchestrator v25.5 is installed on Windows Server 2022 Standard.
- The Universal Orchestrator is approved and checking in to Command.
- EJBCA Enterprise is installed and integrated with Command.
- An issuing CA exists in EJBCA.
- The issuing CA has been added to Keyfactor Command.
- The certificate template has been imported into Command.
- An enrollment pattern exists in Command using that template.
- nginx is installed on Ubuntu and serving a default web site.
- DNS for the nginx server is working.

- A TLS certificate has already been issued for the nginx server.

This guide focuses on the work required to turn those prerequisites into a reliable one-click renewal demo.

---

1. Final Reference Architecture

The final architecture is intentionally staged.

Do not let RFPEM directly manage the live nginx certificate files. Instead, use this pattern:

```
Keyfactor Command
      |
      v
Universal Orchestrator on Windows Server 2022
      |
      | SSH/SFTP
      v
Linux nginx Server
      |
      | RFPEM writes cert/key here
      v
/opt/keyfactor/nginx/staged/
      |
      | promotion script copies staged files into production
      v
/etc/nginx/certs/
/etc/nginx/private/
      |
      | nginx reload
      v
Active HTTPS site
```

Why This Architecture Was Chosen

During implementation, direct RFPEM management of the live nginx certificate file caused repeated renewal conflicts. The error looked like this:

Cannot add a new certificate to a PEM store that already contains a certificate/key entry.

The staged design avoids that problem:

- RFPEM manages only staging files.
- nginx reads only stable production files.

- A controlled script promotes staged files into production.
- nginx is validated before reload.
- The same workflow works repeatedly for demos.

---

2. Definitions

What is a Certificate?

A TLS certificate is a file that helps prove the identity of a server. When a browser connects to nginx over HTTPS, nginx presents its certificate. The browser checks whether the certificate is trusted, valid, and issued for the correct hostname.

What is a Private Key?

The private key is the secret cryptographic material that matches the public key in the certificate. nginx needs both the certificate and private key to serve HTTPS.

Never publish or share private keys.

What is PEM?

PEM stands for Privacy Enhanced Mail. Despite the name, PEM is the most common certificate format used by Linux web servers.

PEM files are Base64-encoded text files that look like this:

-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----

Private key files may look like this:

-----BEGIN PRIVATE KEY-----
...
-----END PRIVATE KEY-----

nginx commonly uses PEM certificate and private key files.

What is RFPEM?

RFPEM is the Keyfactor Remote File Orchestrator certificate store type for PEM files.

RF = Remote File
PEM = PEM certificate format

RFPEM allows a Keyfactor Universal Orchestrator to inventory and manage Linux PEM certificate files over SSH/SFTP without installing a Keyfactor agent directly on the Linux nginx server.

What is Universal Orchestrator?

Universal Orchestrator is the Keyfactor component that performs jobs on behalf of Command. In this reference implementation, the orchestrator runs on Windows Server 2022 and connects to the Linux nginx server over SSH/SFTP.

What is One-Click Renewal?

One-click renewal is a Keyfactor workflow where an operator clicks Renew on a managed certificate and Keyfactor handles issuing the replacement certificate. With private key retention and RFPEM management configured correctly, the renewed certificate can also be deployed automatically.

What is Private Key Retention?

Private key retention means Keyfactor retains deployable private key material so it can later deploy the renewed certificate and key to a managed certificate store. Without private key retention, the renewal may succeed but automated deployment will not have the private key needed by nginx.

Inventory vs Management Jobs

RFPEMInventory jobs read certificate files and update Command inventory.

RFPEMManagement jobs change certificate files, deploy certificates, and run post-job commands.

---

3. Placeholder Values Used in This Guide

Replace these placeholders with your own values.

Placeholder	Meaning	Example
yourcommandserver.domain.com	Keyfactor Command FQDN	command.example.com
yourorchestrator.domain.com	Universal Orchestrator server FQDN	orch01.example.com
yourlinuxserver.domain.com	nginx server FQDN	nginx01.example.com
yourservername.domain.com	certificate common name / FQDN	nginx01.example.com
yourlinuxuser	Linux orchestration account	nginxadmin

Placeholder	Meaning	Example
yourdomain	Your DNS domain	example.com

4. Validate the nginx Server Before Starting

Log into the nginx server.

ssh yourlinuxuser@yourlinuxserver.domain.com

Check Ubuntu details:

hostnamectl

Check nginx:

nginx -v
systemctl status nginx --no-pager

Expected result:

- nginx is installed.
- nginx service is active and running.
- HTTPS already works or will be configured during this guide.

---

5. Install the Remote File Orchestrator Extension

This section is performed on the Windows Universal Orchestrator server.

5.1 Install kfutil

Install kfutil on the orchestrator server following Keyfactor's guidance for your environment.

kfutil is used to create Keyfactor certificate store types from integration manifests.

5.2 Download the Remote File Orchestrator Extension

Download the Remote File Orchestrator extension release that is compatible with your Universal Orchestrator version.

For Universal Orchestrator v25.5, use the appropriate current extension build for that platform.

Extract it to:

C:\Program Files\Keyfactor\Keyfactor Orchestrator\extensions\remote-file-orchestrator

The extension DLLs and manifest.json should be directly inside that folder, not nested one level deeper.

5.3 Download integration-manifest.json

Place integration-manifest.json in the same folder:

C:\Program Files\Keyfactor\Keyfactor Orchestrator\extensions\remote-file-orchestrator\integration-manifest.json

5.4 Create the Store Types

Open PowerShell on the orchestrator server.

```
cd "C:\Program Files\Keyfactor\Keyfactor Orchestrator\extensions\remote-file-orchestrator"
```

Run:

```
kfutil store-types create --from-file .\integration-manifest.json
```

When prompted, provide your Command hostname, authentication type, username, password, API path, and TLS options.

Use this API path unless your environment is customized:

KeyfactorAPI

If your orchestrator server already trusts the Command TLS certificate, leave the CA cert path blank and keep TLS verification enabled.

5.5 Restart the Orchestrator

```
Restart-Service KeyfactorOrchestrator-Default
```

5.6 Verify RFPEM Capability

In Keyfactor Command, open:

Orchestrators -> yourorchestrator

Verify that RFPEM appears as a capability.

If RFPEM does not appear, re-approve or refresh the orchestrator record after the extension is installed and store types are created.

---

6. Configure SSH Authentication

RFPEM uses SSH/SFTP to connect from the Windows orchestrator to the Linux nginx server.

6.1 Generate an SSH Key Pair on the Orchestrator

On the Windows Universal Orchestrator server, open PowerShell as the account you are using for setup.

Run:

```
ssh-keygen -t ed25519
```

Accept the default location.

The key files are usually created here:

C:\Users\<username>\.ssh\id_ed25519
C:\Users\<username>\.ssh\id_ed25519.pub

The .pub file is the public key.

The file without .pub is the private key.

6.2 Display the Public Key

```
type $env:USERPROFILE\.ssh\id_ed25519.pub
```

Copy the entire output line.

6.3 Add the Public Key to the Linux Server

Log into the Linux server as the orchestration user.

```
ssh yourlinuxuser@yourlinuxserver.domain.com
```

Create the .ssh directory:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

Edit authorized_keys:

```
vi ~/.ssh/authorized_keys
```

Press i to enter insert mode.

Paste the public key.

Press ESC.

Type:

```
:wq
```

Press Enter.

Set file permissions:

```
chmod 600 ~/.ssh/authorized_keys
```

6.4 Test Passwordless SSH

From the Windows orchestrator server:

```
ssh yourlinuxuser@yourlinuxserver.domain.com
```

Expected result: you should log in without being prompted for the Linux user's password.

If you are prompted for a password, stop and fix SSH authentication before continuing.

6.5 Capture the SSH Private Key for Keyfactor

Display the private key on the orchestrator server:

```
type $env:USERPROFILE\.ssh\id_ed25519
```

Copy the entire private key, including:

-----BEGIN OPENSSH PRIVATE KEY-----

and:

-----END OPENSSH PRIVATE KEY-----

You will paste this into the RFPEM certificate store's Set Server Password secret value field.

This feels odd because the field says password, but for RFPEM SSH key authentication the field can contain the SSH private key.

---

7. Prepare nginx Certificate Directories

On the nginx server:

```
sudo mkdir -p /etc/nginx/certs
sudo mkdir -p /etc/nginx/private
```

Set permissions:

```
sudo chown root:root /etc/nginx/certs
sudo chown root:root /etc/nginx/private
sudo chmod 755 /etc/nginx/certs
sudo chmod 700 /etc/nginx/private
```

These are the live production paths used by nginx.

---

8. Install the Initial nginx Certificate

You already issued a certificate for the nginx server. This section shows how to install it manually the first time.

Copy the PFX to the Linux server:

```
scp yourservername.domain.com.pfx
yourlinuxuser@yourlinuxserver.domain.com:/home/yourlinuxuser/
```

On the Linux server:

```
mkdir -p ~/certwork
chmod 700 ~/certwork
mv ~/yourservername.domain.com.pfx ~/certwork/
cd ~/certwork
```

Extract the certificate:

```
openssl pkcs12 -in yourservername.domain.com.pfx -clcerts -nokeys -out
yourservername.domain.com.crt
```

Extract the private key:

```
openssl pkcs12 -in yourservername.domain.com.pfx -nocerts -nodes -out
yourservername.domain.com.key
```

Move to the nginx live paths:

```
sudo mv yourservername.domain.com.crt /etc/nginx/certs/
sudo mv yourservername.domain.com.key /etc/nginx/private/
```

Set permissions:

```
sudo chown root:root /etc/nginx/certs/yourservername.domain.com.crt
sudo chown root:root /etc/nginx/private/yourservername.domain.com.key
sudo chmod 644 /etc/nginx/certs/yourservername.domain.com.crt
sudo chmod 600 /etc/nginx/private/yourservername.domain.com.key
```

---

9. Configure nginx for HTTPS

Edit the default nginx site:

```
sudo vi /etc/nginx/sites-available/default
```

Inside the server block, configure HTTPS listeners and certificate paths.

Example:

```
listen 443 ssl default_server;
listen [::]:443 ssl default_server;

ssl_certificate     /etc/nginx/certs/yourservername.domain.com.crt;
ssl_certificate_key /etc/nginx/private/yourservername.domain.com.key;
```

Save and exit.

Validate nginx:

```
sudo nginx -t
```

Reload nginx:

```
sudo systemctl reload nginx
```

Test in a browser:

https://yourlinuxserver.domain.com

---

10. Install the Keyfactor-Branded Demo Web Page

The demo page provides a visual application for the renewal demo. It displays:

- Current certificate serial number
- Current certificate validity period
- Keyfactor-branded layout

The page is installed as:

https://yourlinuxserver.domain.com/keyfactor/

10.1 Copy the Web Page Package to the nginx Server

From your workstation or orchestrator server:

```
scp nginx-keyfactor-cert-status-example.zip
yourlinuxuser@yourlinuxserver.domain.com:/home/yourlinuxuser/
```

On the Linux server:

```
cd ~
unzip nginx-keyfactor-cert-status-example.zip
cd nginx-keyfactor-cert-status-example
```

10.2 Install the Web Page

```
sudo WEB_ROOT=/var/www/html SUBPAGE=keyfactor ./install.sh
```

Expected result:

Installed Keyfactor Command certificate status example to /var/www/html/keyfactor

10.3 Install CGI Support

The web page uses JavaScript to call a CGI endpoint that reads the active nginx certificate and returns JSON.

Install fcgiwrap:

```
sudo apt update
sudo apt install fcgiwrap -y
```

Enable and start fcgiwrap:

```
sudo systemctl enable fcgiwrap
sudo systemctl start fcgiwrap
```

Check status:

```
systemctl status fcgiwrap --no-pager
```

10.4 Create the CGI Directory

```
sudo mkdir -p /var/www/html/cgi-bin
sudo chown root:root /var/www/html/cgi-bin
sudo chmod 755 /var/www/html/cgi-bin
```

10.5 Create cert-info.cgi

```
sudo vi /var/www/html/cgi-bin/cert-info.cgi
```

Press i and paste this script. Replace yourservername.domain.com with your actual certificate filename.

```
#!/usr/bin/env bash
set -euo pipefail

CERT_FILE="/etc/nginx/certs/yourservername.domain.com.crt"

printf 'Content-Type: application/json\r\n'
printf 'Cache-Control: no-store\r\n'
printf '\r\n'

if [ ! -r "$CERT_FILE" ]; then
  printf '{"error":"certificate file is not readable"}\n'
```

```
  exit 0
fi

serial_raw="$(openssl x509 -in "$CERT_FILE" -noout -serial | sed 's/^serial=//')"
not_before="$(openssl x509 -in "$CERT_FILE" -noout -startdate | sed 's/^notBefore=//')"
not_after="$(openssl x509 -in "$CERT_FILE" -noout -enddate | sed 's/^notAfter=//')"
serial_colon="$(echo "$serial_raw" | sed 's/../&:/g; s/:$//')"

printf '{\n'
printf '  "serialNumber": "%s",\n' "$serial_colon"
printf '  "notBefore": "%s",\n' "$not_before"
printf '  "notAfter": "%s"\n' "$not_after"
printf '}\n'
```

Save and exit.

Set permissions:

```
sudo chown root:root /var/www/html/cgi-bin/cert-info.cgi
sudo chmod 755 /var/www/html/cgi-bin/cert-info.cgi
```

10.6 Configure nginx to Serve CGI

Edit the nginx site:

```
sudo vi /etc/nginx/sites-available/default
```

Inside the server block, add:

```
location /cgi-bin/ {
    gzip off;
    root /var/www/html;
    fastcgi_pass unix:/run/fcgiwrap.socket;
    include /etc/nginx/fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
```

Validate nginx:

```
sudo nginx -t
```

Reload nginx:

```
sudo systemctl reload nginx
```

10.7 Test the CGI Endpoint

```
curl -k https://yourlinuxserver.domain.com/cgi-bin/cert-info.cgi
```

Expected output:


```
{
  "serialNumber": "AA:BB:CC:...",
  "notBefore": "May 28 13:50:15 2026 GMT",
  "notAfter": "May 28 13:50:14 2027 GMT"
}
```


10.8 Test the Web Page

Open:

https://yourlinuxserver.domain.com/keyfactor/

You should see the Keyfactor-branded page with certificate serial number and validity period.

---

11. Create the Staging Architecture

Create the RFPEM staging directory:

```
sudo mkdir -p /opt/keyfactor/nginx/staged
```

Set ownership so the RFPEM SSH user can write staged files:

```
sudo chown -R yourlinuxuser:yourlinuxuser /opt/keyfactor/nginx
```

Set permissions:

```
sudo chmod 750 /opt/keyfactor/nginx
sudo chmod 750 /opt/keyfactor/nginx/staged
```

Copy the current live cert/key into staging. This seeds the store so the first inventory operation succeeds.

```
sudo cp /etc/nginx/certs/yourservername.domain.com.crt
/opt/keyfactor/nginx/staged/yourservername.domain.com.crt
sudo cp /etc/nginx/private/yourservername.domain.com.key
/opt/keyfactor/nginx/staged/yourservername.domain.com.key
```

Set ownership and permissions:

```
sudo chown yourlinuxuser:yourlinuxuser
/opt/keyfactor/nginx/staged/yourservername.domain.com.crt
sudo chown yourlinuxuser:yourlinuxuser
/opt/keyfactor/nginx/staged/yourservername.domain.com.key
```

```
sudo chmod 644 /opt/keyfactor/nginx/staged/yourservername.domain.com.crt
sudo chmod 600 /opt/keyfactor/nginx/staged/yourservername.domain.com.key
```

---

12. Create the Promotion Script

The promotion script copies the staged RFPEM files into the live nginx paths, fixes ownership and permissions, validates nginx, and reloads nginx.

Create the script:

```
sudo vi /usr/local/sbin/keyfactor-nginx-deploy
```

Press i and paste this script. Replace yourservername.domain.com with your hostname/certificate filename.

```
#!/bin/bash
set -euo pipefail

STAGED_CERT="/opt/keyfactor/nginx/staged/yourservername.domain.com.crt"
STAGED_KEY="/opt/keyfactor/nginx/staged/yourservername.domain.com.key"

LIVE_CERT="/etc/nginx/certs/yourservername.domain.com.crt"
LIVE_KEY="/etc/nginx/private/yourservername.domain.com.key"

test -s "$STAGED_CERT"
test -s "$STAGED_KEY"

install -o root -g root -m 0644 "$STAGED_CERT" "$LIVE_CERT"
install -o root -g root -m 0600 "$STAGED_KEY" "$LIVE_KEY"

nginx -t >/dev/null 2>&1
systemctl reload nginx
```

Save and exit.

Set permissions:

```
sudo chown root:root /usr/local/sbin/keyfactor-nginx-deploy
sudo chmod 750 /usr/local/sbin/keyfactor-nginx-deploy
```

---

13. Configure Least-Privilege sudo

The orchestrator needs permission to run the promotion script as root, but it should not have broad sudo access.

Create a sudoers file:

```
sudo visudo -f /etc/sudoers.d/keyfactor-nginx
```

Press i and paste:

```
# Keyfactor RFPEM post-job permissions for nginx certificate lifecycle automation.
# Allows only the controlled nginx promotion script.

Cmnd_Alias KEYFACTOR_NGINX_CMDS = /usr/local/sbin/keyfactor-nginx-deploy

yourlinuxuser ALL=(root) NOPASSWD: KEYFACTOR_NGINX_CMDS
```

Save and exit.

Set permissions:

```
sudo chmod 0440 /etc/sudoers.d/keyfactor-nginx
```

Validate sudo configuration:

```
sudo visudo -c
```

Expected result includes:

```
/etc/sudoers.d/keyfactor-nginx: parsed OK
```

Test the script:

```
sudo /usr/local/sbin/keyfactor-nginx-deploy
```

Expected result: no output and no password prompt.

Check exit code:

```
echo $?
```

Expected result:

```
0
```

---

14. Create the nginx-cert-status Validation Script

This script gives you a simple command to run before and after renewals.

Create the script:

sudo vi /usr/local/bin/nginx-cert-status

Press i and paste this script. Replace yourservername.domain.com with your hostname/certificate filename.

```
#!/bin/bash

STAGED_CERT="/opt/keyfactor/nginx/staged/yourservername.domain.com.crt"
ACTIVE_CERT="/etc/nginx/certs/yourservername.domain.com.crt"

fmt_cert_date() {
  TZ=America/Chicago date -d "$(echo "$1" | sed 's/ GMT$/ UTC/')" '+%Y-%m-%d %I:%M:%S %p %Z'
}

echo "==================== SERVER OS ===================="
hostnamectl

echo
echo "==================== NGINX ===================="
nginx -v 2>&1
systemctl status nginx --no-pager | head -n 12

echo
echo "==================== STAGED CERTIFICATE ===================="

STAGED_START="$(openssl x509 -in "$STAGED_CERT" -noout -startdate | sed 's/^notBefore=//')"
STAGED_END="$(openssl x509 -in "$STAGED_CERT" -noout -enddate | sed 's/^notAfter=//')"

openssl x509 -in "$STAGED_CERT" -noout -subject -issuer -serial

echo "Valid From: $(fmt_cert_date "$STAGED_START")"
echo "Valid To  : $(fmt_cert_date "$STAGED_END")"

echo
echo "==================== ACTIVE CERTIFICATE ===================="

ACTIVE_START="$(openssl x509 -in "$ACTIVE_CERT" -noout -startdate | sed 's/^notBefore=//')"
ACTIVE_END="$(openssl x509 -in "$ACTIVE_CERT" -noout -enddate | sed 's/^notAfter=//')"
```

```
openssl x509 -in "$ACTIVE_CERT" -noout -subject -issuer -serial

echo "Valid From: $(fmt_cert_date "$ACTIVE_START")"
echo "Valid To  : $(fmt_cert_date "$ACTIVE_END")"
```

Save and exit.

Set permissions:

```
sudo chown root:root /usr/local/bin/nginx-cert-status
sudo chmod 755 /usr/local/bin/nginx-cert-status
```

Run it:

```
nginx-cert-status
```

Expected result:

- OS details
- nginx status
- staged certificate serial and validity
- active certificate serial and validity
- staged and active serial numbers match

---

15. Update the Remote File Extension Post-Job Command

This step is performed on the Windows Universal Orchestrator server.

The RFPEM certificate store has a post-job dropdown option called NGINX Restart. You will redefine that option so it runs the promotion script.

15.1 Open the Extension Folder

On the Windows orchestrator server, open PowerShell as Administrator.

```
cd "C:\Program Files\Keyfactor\Keyfactor Orchestrator\extensions\remote-file-orchestrator"
```

15.2 Back Up config.json

```
Copy-Item .\config.json .\config.json.bak
```

15.3 Edit config.json

```
notepad .\config.json
```

Locate the PostJobCommands section.

Find or add the Linux NGINX Restart entry:

```
{
  "Name": "NGINX Restart",
  "Environment": "Linux",
  "Command": "sudo /usr/local/sbin/keyfactor-nginx-deploy"
}
```

Save the file.

15.4 Restart the Orchestrator

Restart-Service KeyfactorOrchestrator-Default

Verify the service is running:

Get-Service KeyfactorOrchestrator-Default

---

16. Create an Application in Keyfactor Command

Applications are logical groupings for certificate stores. This step is optional but recommended for a clean demo.

In Keyfactor Command:

Applications -> Add

Use:

Application Name: NGINX-Web

For first-time setup, do not enable an application-level schedule yet unless you want all stores in the application to inherit that schedule.

Save the application.

---

17. Create the RFPEM Certificate Store in Keyfactor Command

This step is performed in Keyfactor Command.

17.1 Navigate to Certificate Stores

Locations -> Certificate Stores -> Add

17.2 Configure the Store

Use these values.

Field	Value
Category	RFPEM
Application	NGINX-Web, if created
Client Machine	yourlinuxserver.domain.com
Store Path	/opt/keyfactor/nginx/staged/ yourservername.domain.com.crt
Orchestrator	yourorchestrator
Set Server Username	yourlinuxuser
Set Server Password	Full SSH private key contents
Linux File Permissions on Store Creation	644
Linux File Owner on Store Creation	yourlinuxuser:yourlinuxuser
Sudo Impersonating User	root
Trust Store	false
Store Includes Chain	true
Separate Private Key File Location	/opt/keyfactor/nginx/staged/ yourservername.domain.com.key
Ignore Private Key On Inventory	false
Remove Root Certificate From Chain	true
Include Port in SPN for WinRM	false
SSH Port	22
Use Shell Commands	true
Post Job Application Restart	NGINX Restart
Use SSL	false
Set Store Password	No Value / blank
Create Certificate Store If Missing	checked if available
Inventory Schedule	Immediate for first setup

17.3 How to Enter Server Username

When selecting Set Server Username, Keyfactor may open a secret value dialog.

Do not select No Value.

Select:

Load From Keyfactor Secrets

Enter:

yourlinuxuser

in both secret value fields.

17.4 How to Enter Server Password / SSH Private Key

When selecting Set Server Password, choose:

Load From Keyfactor Secrets

Paste the entire SSH private key from the orchestrator server into both fields.

It must include:

-----BEGIN OPENSSH PRIVATE KEY-----

and:

-----END OPENSSH PRIVATE KEY-----

Do not paste the public key here.

Do not paste only part of the private key.

Save the store.

---

18. Run Initial RFPEM Inventory

After creating the store, run or schedule an immediate inventory.

In Command:

Locations -> Certificate Stores -> Select your RFPEM store -> Schedule Inventory

Expected result:

- RFPEMInventory job runs.
- The staged certificate is discovered.
- The certificate appears in the store inventory.

If inventory fails with No such file, verify the staged cert and key filenames match the RFPEM configuration exactly.

If inventory fails with Permission denied, verify SSH key authentication and file permissions.

---

19. Enable One-Click Renewal and Private Key Retention

In Keyfactor Command, ensure the enrollment pattern/template used for this certificate supports one-click renewal.

Enable:

One-click renewal

Enable:

Private key retention

For a demo lab, a simple retention policy is:

Indefinite

Why this matters:

nginx requires both a certificate and private key. During automated deployment, Keyfactor must be able to deploy the private key. Without private key retention, renewal may succeed but RFPEM deployment may not have the private key needed to update nginx.

Certificate cleanup can also be enabled if you want old certificates cleaned up according to policy, but do not aggressively revoke old certificates until you have confirmed the renewed certificate is active.

---

20. Perform the First One-Click Renewal Test

Important:

Perform renewals from the RFPEM certificate store inventory view, not from the global certificate inventory view.

In Command:

Locations -> Certificate Stores -> Select RFPEM Store -> View Inventory

Select the managed certificate.

Click:

Renew

Watch for jobs:

RFPEMManagement
RFPEMInventory

Expected workflow:

Keyfactor renews the certificate
RFPEM deploys cert/key to staging
Post-job action runs NGINX Restart

NGINX Restart executes keyfactor-nginx-deploy
The script promotes staged files into live nginx paths
nginx reloads
Inventory reconciles
The web page shows new certificate metadata

---

21. Validate the Renewal

On the Linux server:

nginx-cert-status

Verify:

- Staged certificate serial number changed.
- Active certificate serial number changed.
- Staged and active serial numbers match.
- Validity period updated.

Open the demo page:

https://yourlinuxserver.domain.com/keyfactor/

Verify the page shows the new serial number and new validity period.

If the web page does not update immediately, refresh the page. The JavaScript refreshes certificate data periodically and bypasses cache.

---

22. Recommended Inventory Schedule

After validation, configure:

Store-specific RFPEMInventory: hourly
Global/orchestrator RFPEMInventory: daily

Inventory jobs are read-only. They reconcile what is actually on disk with what Keyfactor Command shows in inventory.

---

23. Demo Procedure

Use this flow during your demo.

Before Renewal

On the Linux server:

nginx-cert-status

Open:

https://yourlinuxserver.domain.com/keyfactor/

Show:

- current serial number
- current validity period

Perform Renewal

In Command:

Locations -> Certificate Stores -> RFPEM Store -> View Inventory -> Renew

Watch the RFPEMManagement job complete.

After Renewal

Run:

nginx-cert-status

Refresh the browser page.

Show:

- new serial number
- new validity period
- staged and active certs match

---

24. Troubleshooting

Permission denied (password)

Cause:

RFPEM tried password auth or could not use the SSH key.

Fix:

- Confirm public key is in ~/.ssh/authorized_keys on Linux.
- Confirm private key was pasted into Set Server Password.

- Confirm username is correct.
- Test SSH from orchestrator to Linux.

No such file

Cause:

RFPEM path does not match the actual file path.

Fix:

```
ls -l /opt/keyfactor/nginx/staged
```

Compare filenames with RFPEM Store Path and Separate Private Key File Location.

Cannot add a new certificate to a PEM store that already contains a certificate/key entry

Cause:

RFPEM is trying to add into an existing live PEM store.

Fix:

Use the staged architecture in this guide. Do not point RFPEM directly at live nginx certificate files.

Certificate deployed but web page still shows old certificate

Cause:

nginx may not have reloaded, browser cache may need refresh, or the CGI endpoint is reading the wrong certificate path.

Fix:

```
sudo nginx -t
sudo systemctl reload nginx
nginx-cert-status
curl -k https://yourlinuxserver.domain.com/cgi-bin/cert-info.cgi
```

RFPEM inventory shows old certificate after successful deployment

Cause:

Inventory has not run yet.

Fix:

Run or schedule RFPEMInventory for the store.

Post-job warning mentioning nginx -t output

Cause:

Some command output may be interpreted as a warning.

Fix:

The promotion script redirects successful nginx -t output to /dev/null:

```
nginx -t >/dev/null 2>&1
```

---

25. Final Success Criteria

You are done when all of the following are true:

- RFPEM capability exists on the orchestrator.
- SSH from orchestrator to nginx works.
- RFPEM inventory succeeds.
- RFPEMManagement succeeds.
- One-click renewal succeeds.
- New certificate is deployed to staging.
- Promotion script copies the new cert/key to live nginx paths.
- nginx reloads successfully.
- Browser shows the renewed certificate.
- nginx-cert-status shows staged and active serial numbers match.
- The Keyfactor-branded /keyfactor/ web page shows the updated certificate details.

---

26. Key Lessons Learned

1. Do not point RFPEM directly at live nginx certificate files for repeatable one-click renewal demos.
2. Use a staging path managed by RFPEM.
3. Use a local promotion script to copy staged files into production.
4. Use least-privilege sudo for the promotion script only.
5. Renew from the RFPEM store inventory view, not from global certificate inventory.
6. Private key retention is required for automated nginx certificate deployment.
7. Inventory and management jobs are different and both matter.
8. A simple web page that displays certificate serial and validity makes the demo easy to understand.